

# GreyNOC Field Manual

## Creating a Simple P2P PGP Node Network

Using Termux, Wi-Fi, Bluetooth Fallback, and Store-and-Forward Relay Tactics

Public Training Edition | Version 0.1 | Prepared for GreyNOC

### Use statement

This guide is for lawful, authorized, privacy-focused training, emergency readiness, and local file transfer between devices you own or are permitted to use. It is not a guide for unauthorized access, evasion, data theft, or misuse.

## Contents

- 1. Purpose and Scope
- 2. Safety, Legal, and Trust Model
- 3. Architecture Overview
- 4. Required Tools
- 5. Node Folder Layout
- 6. PGP Identity Setup
- 7. Public Key Exchange
- 8. Encrypting, Signing, and Decrypting Files
- 9. Wi-Fi Hotspot Node Network
- 10. SSH and SCP Transfers
- 11. Temporary Pull Transfer with Python HTTP Server
- 12. Bluetooth Fallback
- 13. Store-and-Forward Relay Method
- 14. Naming, Hashing, and Manifests
- 15. Security Hygiene and Lost Device Response
- 16. Two-Phone Field Test
- 17. Limits and Design Notes
- 18. First QA/QC Review Log
- 19. Source Notes

## 1. Purpose and Scope

This manual explains how to build a simple peer-to-peer node workflow using Android devices, Termux, Wi-Fi hotspot or shared Wi-Fi, Bluetooth file sharing fallback, SSH/SCP, and OpenPGP encryption through GnuPG.

The design goal is practical field communication, not a fully automated routing mesh. Each node can encrypt files, exchange public keys, transfer ciphertext over local links, and relay encrypted files without being able to read them.

### Core principle

The network path is not the security boundary. Encrypt first, verify keys, transfer ciphertext, and decrypt only at the intended destination.

## 2. Safety, Legal, and Trust Model

Use this workflow only with devices, networks, keys, and accounts you own or have explicit permission to operate. Do not use it for unauthorized access, hidden exfiltration, impersonation, harassment, or illegal activity.

Approved examples include emergency readiness, family or team file sharing, cybersecurity lab training, disaster recovery communications, local incident response coordination, and privacy-focused document transfer between trusted participants.

The trust model assumes that local networks, relays, Bluetooth, and temporary file servers may be observed or mishandled. Therefore, files should be encrypted and, when practical, signed before being transferred.

## 3. Architecture Overview

Each Android device acts as a node. Nodes may originate, relay, or receive encrypted files.

| Role             | Function                                          | Can read message contents?           |
|------------------|---------------------------------------------------|--------------------------------------|
| Origin node      | Creates, signs, encrypts, and sends files.        | Yes, before encryption.              |
| Relay node       | Forwards encrypted files to another node.         | No, if encrypted to final recipient. |
| Destination node | Receives and decrypts files with its private key. | Yes, after decryption.               |

Example path: Alpha encrypts a file to Charlie, sends it to Bravo, and Bravo forwards the ciphertext to Charlie. Bravo does not need Charlie's private key and should not be able to decrypt the file.

## 4. Required Tools

Install Termux from a trusted source such as F-Droid. Termux uses its own package ecosystem and the pkg helper for installation and updates.

```
pkg update && pkg upgrade -y
pkg install gnupg openssh python rsync nano nmap termux-api qrencode -y
```

Verify core tools:

```
gpg --version
ssh -V
python --version
```

### Android limitation

Termux can run useful networking and cryptography tools, but stock Android limits direct control over some radios and system networking features. Treat Wi-Fi hotspot/shared Wi-Fi as the primary practical transport and Bluetooth as a fallback file-transfer method.

## 5. Node Folder Layout

```
mkdir -p ~/greynoc-node/{inbox,outbox,relay,keys,logs,tmp}
chmod 700 ~/greynoc-node
chmod 700 ~/greynoc-node/*
```

| Folder                | Purpose                                    |
|-----------------------|--------------------------------------------|
| ~/greynoc-node/inbox  | Received encrypted files.                  |
| ~/greynoc-node/outbox | Encrypted files ready to send.             |
| ~/greynoc-node/relay  | Ciphertext queued for forwarding.          |
| ~/greynoc-node/keys   | Public keys, fingerprints, and node cards. |
| ~/greynoc-node/logs   | Manual transfer notes.                     |
| ~/greynoc-node/tmp    | Temporary working area.                    |

## 6. PGP Identity Setup

Create one PGP identity per node or operator. Use clear node names and keep the private key protected with a strong passphrase.

```
gpg --full-generate-key
```

Recommended public training settings: RSA 4096 or modern ECC, expiration of one year, a recognizable node name, and a strong passphrase.

```
gpg --list-keys
gpg --list-secret-keys
gpg --armor --export alpha@greynoc.local > ~/greynoc-node/keys/alpha-public.asc
```

### Private key rule

Never share private keys. Only share public keys and fingerprints.

## 7. Public Key Exchange

Import another node's public key, then verify its fingerprint using a trusted out-of-band method such as in-person comparison, voice call, QR display, or a previously trusted channel.

```
gpg --import ~/greynoc-node/keys/bravo-public.asc
gpg --fingerprint bravo@greynoc.local
```

After verification, set trust locally if appropriate:

```
gpg --edit-key bravo@greynoc.local
# inside the GPG prompt:
trust
save
```

## 8. Encrypting, Signing, and Decrypting Files

Encrypt text or binary files before transfer. ASCII-armored files are easier to inspect and copy, while binary .pgp files are smaller.

```
echo "GreyNOC field test message" > message.txt
gpg --armor --encrypt --recipient bravo@greynoc.local --output message.txt.asc message.txt
```

For authenticity, sign and encrypt:

```
gpg --armor --sign --encrypt --recipient bravo@greynoc.local --output message.signed.asc message.txt
```

Decrypt on the receiving node:

```
gpg --output decrypted-message.txt --decrypt message.signed.asc
```

## 9. Wi-Fi Hotspot Node Network

The simplest field mode is one Android device creating a hotspot while the other devices join it. Internet access is not required for local file transfer.

Suggested setup: create a Wi-Fi hotspot named GreyNOC-FieldNet with a strong WPA2/WPA3 password. Other nodes join the hotspot through normal Android Wi-Fi settings.

```
ip addr
# or, if net-tools is installed:
ifconfig
```

Look for an address such as 192.168.x.x, 172.20.x.x, or another private local address. The exact range depends on the Android device and hotspot implementation.

## 10. SSH and SCP Transfers

Start the SSH server on the receiving Termux node. Termux SSH commonly listens on port 8022.

```
sshd
whoami
passwd
ss -tulpn | grep ssh
```

Send an encrypted file from the sender to the receiver:

```
scp -P 8022 message.signed.asc u0_a123@192.168.43.101:~/greynoc-node/inbox/
```

Replace the username, IP address, and file name for your environment.

## 11. Temporary Pull Transfer with Python HTTP Server

A temporary Python HTTP server can help a receiver pull encrypted files. Only serve ciphertext, never plaintext.

```
cd ~/greynoc-node/outbox
python -m http.server 8080
```

On the receiving node:

```
wget http://192.168.43.100:8080/message.signed.asc -O ~/greynoc-node/inbox/message.signed.asc
```

Stop the server with CTRL+C immediately after the transfer.

## 12. Bluetooth Fallback

Bluetooth should be treated as a fallback transfer method because stock Android and Termux do not provide the same practical command-line Bluetooth networking control as a full Linux system.

Recommended Bluetooth workflow: encrypt in Termux, copy the encrypted file to Android shared storage, use Android's normal share menu, then copy the received file back into Termux for decryption.

```
termux-setup-storage
cp message.signed.asc ~/storage/downloads/
```

On the receiving phone, after Bluetooth receipt:

```
cp ~/storage/downloads/message.signed.asc ~/greynoc-node/inbox/
gpg --decrypt ~/greynoc-node/inbox/message.signed.asc
```

### 13. Store-and-Forward Relay Method

Store-and-forward relay allows nodes that cannot directly reach each other to pass encrypted files through an intermediate node.

```
gpg --armor --sign --encrypt --recipient charlie@greynoc.local --output alpha-to-charlie.asc
message.txt
scp -P 8022 alpha-to-charlie.asc bravo@192.168.43.101:~/greynoc-node/relay/
```

Bravo forwards the same encrypted file to Charlie without decrypting it:

```
scp -P 8022 ~/greynoc-node/relay/alpha-to-charlie.asc charlie@192.168.43.105:~/greynoc-node/inbox/
```

### 14. Naming, Hashing, and Manifests

Use consistent file names so nodes can track messages without opening them.

```
YYYYMMDD-HHMM-FROM-TO-TYPE-ID.asc
20260617-1430-alpha-charlie-sitrep-001.asc
```

Create and verify a hash for each encrypted file:

```
sha256sum message.signed.asc > message.signed.asc.sha256
sha256sum -c message.signed.asc.sha256
```

Suggested relay manifest fields: file name, origin node, destination node, relay node, date/time, transport, encryption status, signature status, SHA256 hash, and notes.

### 15. Security Hygiene and Lost Device Response

| Do                                                                                                                                          | Avoid                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------|
| Encrypt before transfer; verify fingerprints; use strong device unlock; use key expiration; stop temporary servers; rotate keys after loss. | Sharing private keys; trusting unknown public keys; hosting plaintext files; weak hotspot passwords; leaving SSH open when not needed. |

Lost device response: notify operators, stop trusting the affected key, rotate group keys, change hotspot and SSH passwords, remove the lost node from relay paths, and reissue the trusted public key list.

```
gpg --output revoke-alpha.asc --gen-revoke alpha@greynoc.local
```

Generate revocation certificates before field use and store them offline.

### 16. Two-Phone Field Test

This drill confirms that two Termux nodes can exchange public keys, encrypt a file, transfer it locally, and decrypt it.

```
# On both phones
pkg update && pkg install gnupg openssh python nmap -y
mkdir -p ~/greynoc-node/{inbox,outbox,relay,keys,logs,tmp}
gpg --full-generate-key
sshd
```

Exchange public keys. Then on Phone A:

```
gpg --import bravo-public.asc
echo "Test from Alpha to Bravo" > test.txt
gpg --armor --sign --encrypt --recipient bravo@greynoc.local --output test-alpha-bravo.asc test.txt
scp -P 8022 test-alpha-bravo.asc USERNAME@PHONE_B_IP:~/greynoc-node/inbox/
```

On Phone B:

```
gpg --decrypt ~/greynoc-node/inbox/test-alpha-bravo.asc
```

## 17. Limits and Design Notes

This manual describes a simple node workflow, not a fully autonomous dynamic mesh. Stock Android Wi-Fi Direct is available through Android application APIs, but multi-hop mesh-like behavior usually requires application-layer routing or forwarding logic. Termux alone should not be treated as a complete Wi-Fi Direct mesh controller.

Bluetooth is useful for small encrypted files, fingerprints, or public keys, but it is not the preferred primary transport for this workflow. Wi-Fi hotspot or shared Wi-Fi plus SSH/SCP is usually easier to test and operate.

For larger deployments, consider developing a dedicated Android application or using established mesh/off-grid communications projects designed for multi-hop routing, identity management, and message queues.

## 18. First QA/QC Review Log

| Check                  | Result    | Notes                                                                                 | Status |
|------------------------|-----------|---------------------------------------------------------------------------------------|--------|
| Public-facing tone     | Rewritten | Removed rough draft language and framed as lawful public training.                    | Pass   |
| Safety review          | Completed | Added use statement, approved uses, and misuse boundaries.                            | Pass   |
| Technical accuracy     | Reviewed  | Clarified Termux, Android Wi-Fi Direct, Bluetooth, PGP, and manual relay limitations. | Pass   |
| Command sanity         | Reviewed  | Commands checked for obvious syntax issues and Termux path assumptions.               | Pass   |
| Formatting consistency | Reviewed  | Consistent headings, folder names, command blocks, and node terms.                    | Pass   |
| Known limitation       | Open      | No live device lab test was performed in this environment.                            | Note   |

## 19. Source Notes

This public edition was checked against the following public references:

- Termux Wiki - Package Management: [https://wiki.termux.com/wiki/Package\\_Management](https://wiki.termux.com/wiki/Package_Management)
- Termux Wiki - Termux:API: <https://wiki.termux.com/wiki/Termux:API>
- Android Developers - Wi-Fi Direct P2P overview:  
<https://developer.android.com/develop/connectivity/wifi/wifip2p>
- Android Developers - Create P2P connections with Wi-Fi Direct:  
<https://developer.android.com/develop/connectivity/wifi/wifi-direct>
- GnuPG Documentation and Operational GPG Commands: <https://www.gnupg.org/documentation/>